

PROJECT REPORT OF CO-OP PROJECT AT INDUSTRY (MODULE-2)

ON

Detecting and Classifying AI-Generated Source Code Using Feature-Based Machine Learning Techniques

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Nutan Bora

2210992005

Supervised By:

Dr. Isha Kansal

Associate Professor, Chitkara University

Department of Computer Science & Engineering



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Detecting and Classifying AI-Generated Source Code Using Feature-Based Machine Learning Techniques**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Isha Kansal. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Nutan Bora

Date: 01-05-2026

2210992005

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Isha Kansal

Associate Professor, Chitkara University

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behaviour and acts during the course of study.

We express our sincere gratitude to all for providing mean opportunity to undergo co-op project at industry (module-2) as the part of the curriculum.

We are thankful to Dr. Isha Kansal for her support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

We also extend our sincere appreciation to Dr. Isha Kansal who provided her valuable suggestions and precious time in accomplishing our co-op project report.

Lastly, we would like to thank the almighty and our parents for their moral support and friends with whom we share dour day-to day experience and received lots of suggestions that improve our quality of work.

Nutan Bora

ID No. 2210992005

LIST OF FIGURES AND TABLES

LIST OF TABLES-

<u>TABLE NO.</u>	<u>TITLE</u>	<u>PAGE NO.</u>
Table: 2.3.1	Significance of the Project	8
Table: 3.1.1	Hardware Requirements	10
Table: 3.1.2	Software Requirements	10
Table: 4.2.1	System Architecture	11
Table: 4.3.1	Functional Modules Overview	11
Table: 5.1	Tools and Technologies	13
Table: 6.2.1	Feature Extraction Logic	13
Table: 6.3.1	Database Design	13
Table: 7.1	Major Outcomes / Results	15

LIST OF FIGURES-

<u>FIGURE NO.</u>	<u>TITLE</u>	<u>PAGE NO.</u>
Fig.: 4.4.1	Workflow Diagram	12
Fig.: 4.4.2	Workflow Diagram	12
Fig.: 6.4.1	GUI Screenshot- Dashboard	14
Fig.: 6.4.2	GUI Screenshot- Output / Result Page	14
Fig.: 6.4.3	GUI Screenshot- Model Performance	15
Fig.: 6.4.4	GUI Screenshot- About	15

CONTENTS

<u>S.L. NO.</u>	<u>TITLE</u>	<u>PAGE NO.</u>
1.	ABSTRACT	6
2.	INTRODUCTION	7
	2.1 Background and Context	7
	2.2 Problem Statement	7
	2.3 Significance of the Project	8
	2.4 Objectives	9
	2.5 Scope and Limitations	9
3.	SYSTEM REQUIREMENTS	10
	3.1 Hardware Requirements	10
	3.2 Software Requirements	10
4.	METHODOLOGY	10
	4.1 Technical Approach	10
	4.2 System Architecture	11
	4.3 Functional Modules Overview	11
	4.4 Workflow Diagram	12
5.	TOOLS AND TECHNOLOGY	13
6.	IMPLEMENTATION	13
	6.1 Frameworks and Libraries	13
	6.2 Feature Extraction Logic	13
	6.3 Database Design	13
	6.4 GUI Screenshots	14 - 15
7.	MAJOR OUTCOMES / RESULTS	16
8.	CONCLUSION	16
9.	FUTURE SCOPE	17
10.	REFERENCES	17

1. ABSTRACT

The proliferation of Large Language Models (LLMs) has revolutionized software synthesis, enabling the rapid generation of high-quality, functional code. However, this shift introduces a critical *authenticity gap* in both academia and industry. Traditional plagiarism detection tools, which rely on textual similarity, are largely ineffective against AI-generated outputs.

- **The “Perfect Code” Paradox:** AI-generated scripts are typically syntactically perfect and structurally unique, yet they lack the idiosyncratic “noise”, inconsistent formatting, and non-linear logic characteristic of human authorship.
- **The Integrity Crisis:** Educational institutions face difficulty in fairly evaluating student work, while organizations struggle to verify code ownership and originality in increasingly AI-assisted development environments.

The proposed solution introduces a multi-stage automated pipeline designed to quantify the “*digital DNA*” of source code. The architecture is built upon a modular engine that ingests raw code and subjects it to high-fidelity static analysis.

The Logic: “The Perfection of Machines vs. The Noise of Humans”

The system operates on the principle that AI and humans leave different “scuff marks” on their work:

- **The AI Signature (Symmetry):** AI-generated code is uncannily perfect. Machine-generated programs typically exhibit high formatting uniformity that includes perfect indentation, consistent spacing around operators, predictable commenting patterns, and standardized brace placement.
- **The Human Signature (Variability):** Human code is noisy and idiosyncratic. It contains inconsistent blank lines, varied commenting habits, and unique structural “detours” that reflect a person’s individual thought process and problem-solving evolution.

2. INSTRUCTION

2.1 Background and Context

The shift toward AI-assisted development has led to the “*Perfect Code*” Paradox: machine-generated scripts are often structurally flawless, making them indistinguishable from high-quality human code to the naked eye. However, humans typically exhibit non-linear logic and inconsistent formatting, which this project identifies as a “*Human Signature*”. AI-powered coding assistants are now widely used in both academic and industrial environments to improve productivity and reduce development time. However, this transformation introduces challenges in verifying the originality and authorship of code.

2.2 Problem Statement

There is currently no reliable and standardized mechanism to distinguish between AI-generated and human-written source code. The increasing use of AI-powered coding tools has made it difficult to accurately identify the true origin of code, as AI-generated programs are often syntactically correct, logically complete, and structurally unique.

This limitation creates critical challenges in multiple areas, including:

- Academic integrity and fair evaluation
- Authorship attribution and code ownership verification
- Plagiarism detection beyond similarity-based methods
- Responsible AI usage in programming environments
- Originality verification of submitted work
- Software authenticity and trust in codebases

The inability to effectively determine whether code is human-written or AI-generated leads to ambiguity in evaluation processes, reduces transparency, and undermines trust in both academic assessments and professional software development practices.

2.3 Significance of the Project

<u>ASPECT</u>	<u>SIGNIFICANCE</u>
<i>Academic Integrity</i>	Helps maintain fairness in programming evaluations by enabling detection of AI-generated code, ensuring that student performance reflects genuine effort.
<i>Authorship Attribution</i>	Provides a mechanism to verify whether code is written by a human or generated by AI, improving clarity in code ownership.
<i>Plagiarism Detection</i>	Goes beyond traditional similarity-based methods by identifying structural and stylistic patterns instead of relying only on code matching.
<i>Responsible AI Usage</i>	Encourages ethical use of AI tools in programming by discouraging misuse and over-reliance on automated code generation.
<i>Originality Verification</i>	Helps validate whether submitted code is original or AI-assisted, reducing ambiguity in evaluation processes.
<i>Software Authenticity</i>	Ensures trust and reliability in codebases by enabling verification of the origin of code contributions.
<i>Software Forensics</i>	Introduces a structured approach to analysing coding behaviour and stylometric patterns for authorship identification.
<i>Scalable and Interpretable Solution</i>	Offers a system that can be extended to academic platforms, code review tools, and enterprise environments with clear and explainable outputs.

Table: 2.3.1

2.4 Objectives

This project seeks to engineer an intelligent classification framework capable of distinguishing between human-authored and machine-generated source code.

Core Goals:

- Develop a reliable AI code detection system
- Identify measurable coding patterns
- Ensure interpretability of classification results

Key Challenges:

- High similarity in logical correctness between AI and human code
- Variability in human coding styles
- Standardized patterns in AI-generated outputs
- Overcoming the "Black Box" nature of AI which is lack of transparency in how AI models generate outputs.

2.5 Scope and Limitations

Scope:

- a) Works on source code (Python)
- b) Uses static code analysis
- c) Applies ML-based classification

Limitations:

- a) Performance depends on dataset quality
- b) May not detect heavily modified AI code

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

<u>COMPONENT</u>	<u>SPECIFICATION</u>
Processor	Intel i5 or above
RAM	Minimum 8 GB
Storage	256 GB
Internet Connectivity	Required for dataset access, updates, and research resources

Table: 3.1.1

3.2 Software Requirements

<u>SOFTWARE</u>	<u>SPECIFICATION</u>	<u>PURPOSE</u>
Operating System	Windows 10/11, MacOS, or Linux	Platform for development and execution
IDE	VS Code / Jupyter Notebook	Development environment
Language	Python 3.9+	Core development

Table: 3.1.2

4. METHODOLOGY

4.1 Technical Approach

The system employs *Static Program Analysis* combined with *Supervised Machine Learning*. Instead of treating code as plain text, the system treats it as a structured entity, extracting features from both the raw text (stylometric) and the logical skeleton (AST). The workflow follows a rigorous pipeline:

Data Ingestion → AST Parsing → Feature Vectorization → Model Prediction / Classification

System Workflow:

Input: Source code file or text

Processing: Feature extraction and analysis Output:

Output: Classification (AI-generated / Human-written)

4.2 System Architecture

The architecture is divided into three functional pillars that transform raw source code into actionable forensic data. This modular approach ensures that both the stylistic “scuff marks” of human typing and the logical patterns of machine generation are captured.

<u>LAYER</u>	<u>SUB-MODULE</u>	<u>DESCRIPTION</u>	<u>PURPOSE</u>
<i>Sensory Layer</i>	Multi-Dimensional Feature Extractor	Measures granular textual details like indentation entropy and whitespace variance.	Captures the “noise” and formatting inconsistencies of human authorship.
<i>Structural Layer</i>	AST Structural Evaluator	Maps the skeleton of the code using Abstract Syntax Trees to analyse logical flow.	Identifies standardized structural patterns and logical symmetry in AI code.
<i>Brain Layer</i>	Supervised ML Classifier	Compares extracted features against known datasets (e.g., via Random Forest).	Generates an Interpretative Origin Profile to predict the code's source.

Table: 4.2.1

4.3 Functional Modules Overview

<u>FEATURE</u>	<u>FUNCTION</u>	<u>PURPOSE</u>
Preprocessing	Clean and normalize input code	Ensure consistent input format
Code Parsing	Analyse code structure	Extract meaningful data
Feature Extraction	Identify patterns (indentation, comments, tokens)	Capture coding style
Structural Analysis	Evaluate AST and complexity	Understand code organization
Classification Engine	Apply ML model	Predict code origin
Result Interpretation	Highlight contributing features	Improve transparency
Logging System	Store analysis records	Ensure traceability

Table: 4.3.1

4.4 Workflow Diagram

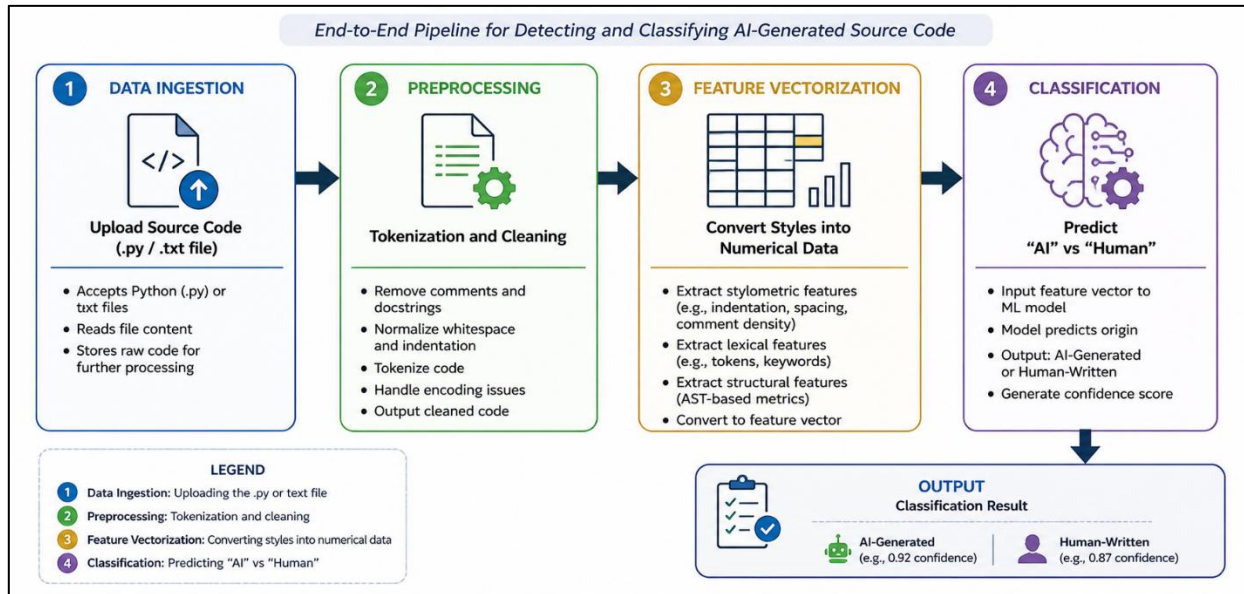


Fig.: 4.4.1

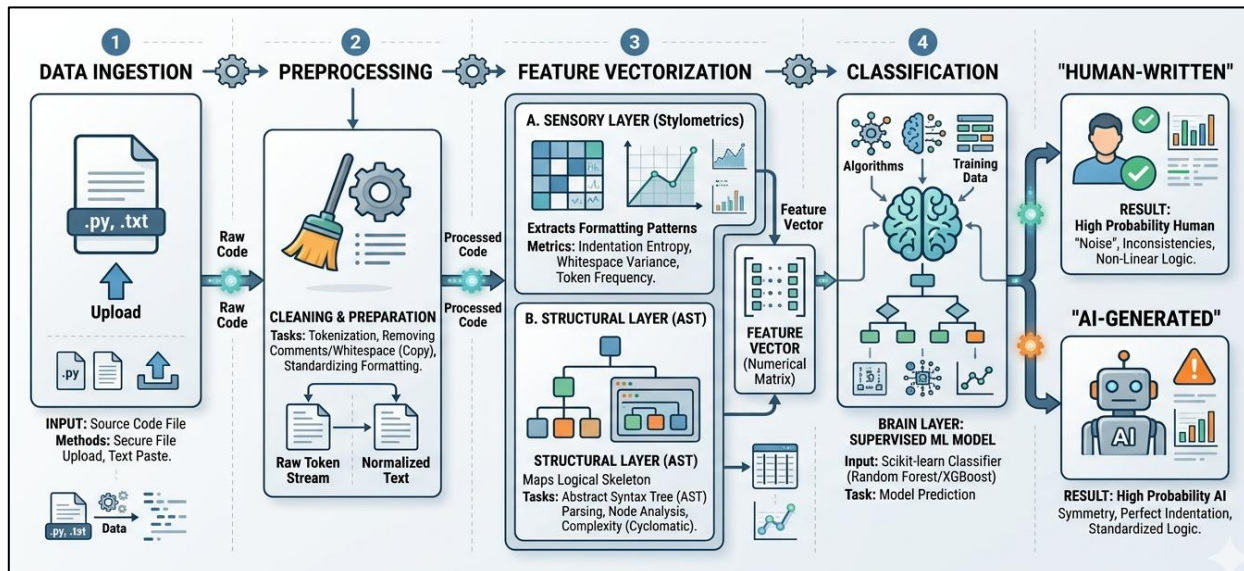


Fig.: 4.4.2

Workflow Diagram Breakdown:

The process follows a linear, high-fidelity pipeline:

- Data Ingestion: Uploading the .py or text file.
- Preprocessing: Tokenization and cleaning.
- Feature Vectorization: Converting styles into numerical data.
- Classification / Model Prediction: Predicting "AI" vs "Human".

5. TOOLS AND TECHNOLOGY

<u>TOOLS</u>	<u>USAGE</u>
Python	Used for core system development and implementation
Scikit-learn	Used for training and applying machine learning models
AST (Abstract Syntax Tree)	Used for parsing and analysing code structure
Pandas	Used for data handling and feature organization
NumPy	Used for numerical computations and feature processing
IDE (VS Code / Jupyter)	Used for coding, testing, and development

Table: 5.1

6. IMPLEMENTATION

6.1 Frameworks and Libraries

- a) Scikit-learn
- b) Pandas
- c) NumPy
- d) AST

6.2 Feature Extraction Logic

<u>FEATURE CATEGORY</u>	<u>DESCRIPTION</u>	<u>METRIC</u>
Stylometric	Formatting habits	Indentation entropy, comment density.
Structural	Logic flow	AST node depth, function complexity.
Symmetry	Uniformity	Consistency of spacing around operators.

Table: 6.2.1

6.3 Database Design

<u>FIELD</u>	<u>DESCRIPTION</u>
Code Input	Stores the raw source code provided for analysis
Features	Contains extracted structural and stylistic patterns from the code
Predictions	Stores the classification result (AI-generated or Human-written)

Table: 6.3.1

6.4 GUI Screenshots

a) Dashboard – Input (Code sample)

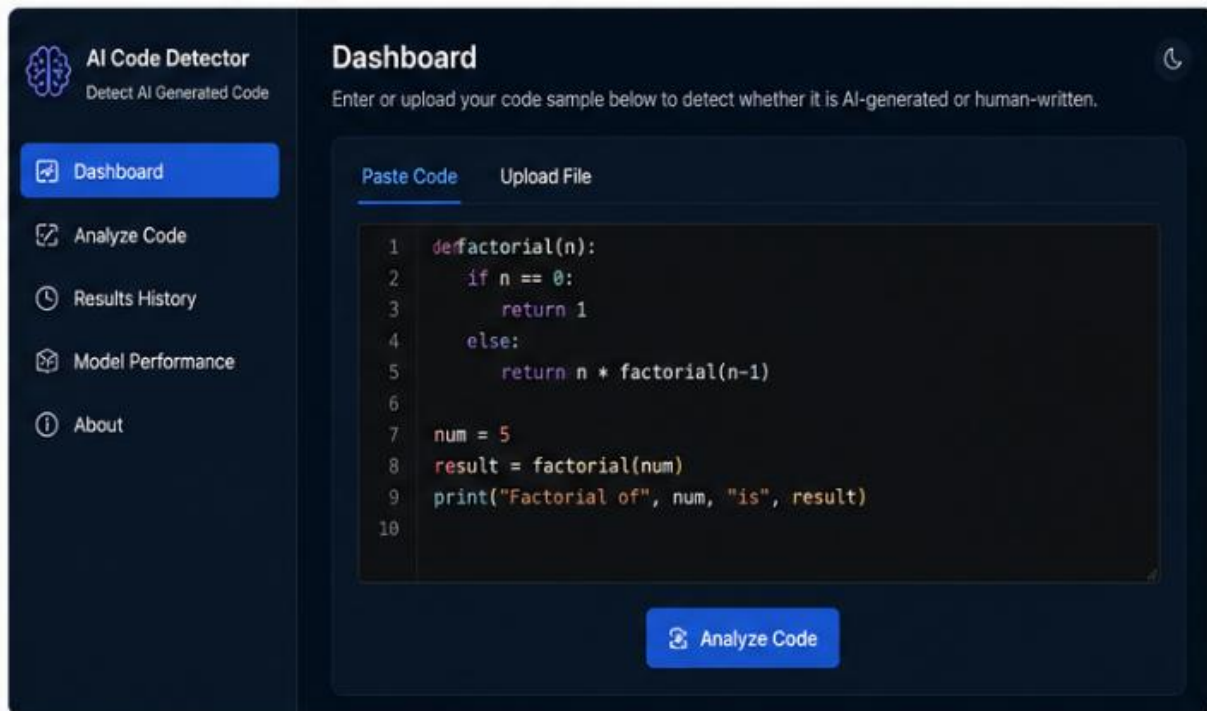


Fig.: 6.4.1

b) Output – Result Page

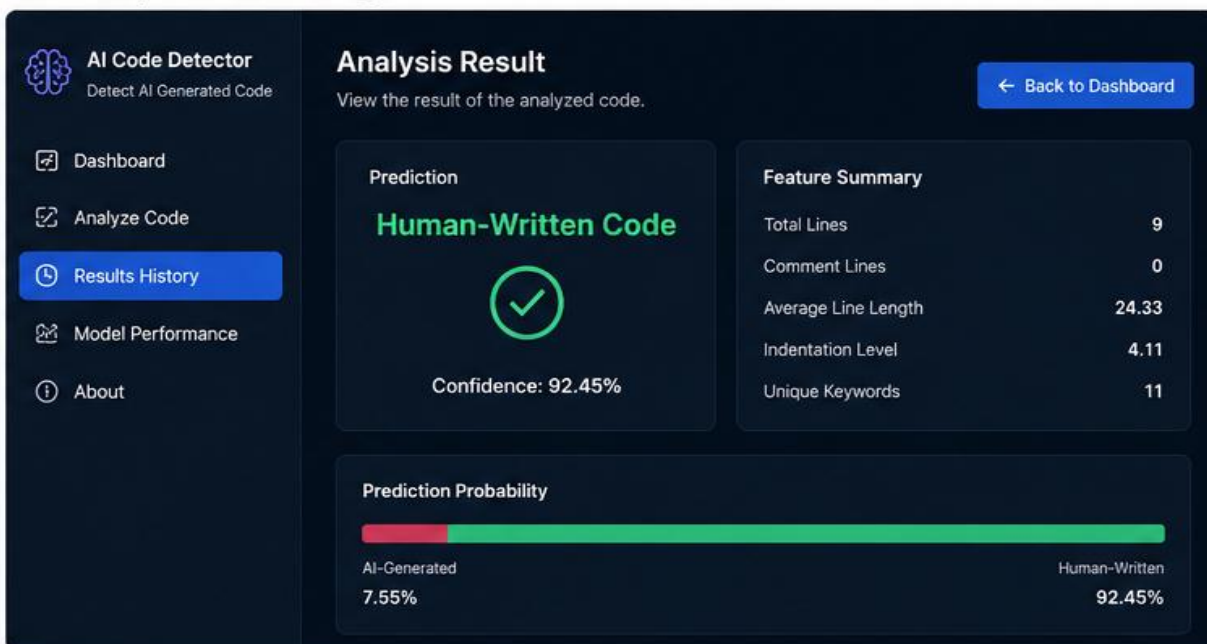


Fig.: 6.4.2

c) Model Performance



Fig.: 6.4.3

d) About Page

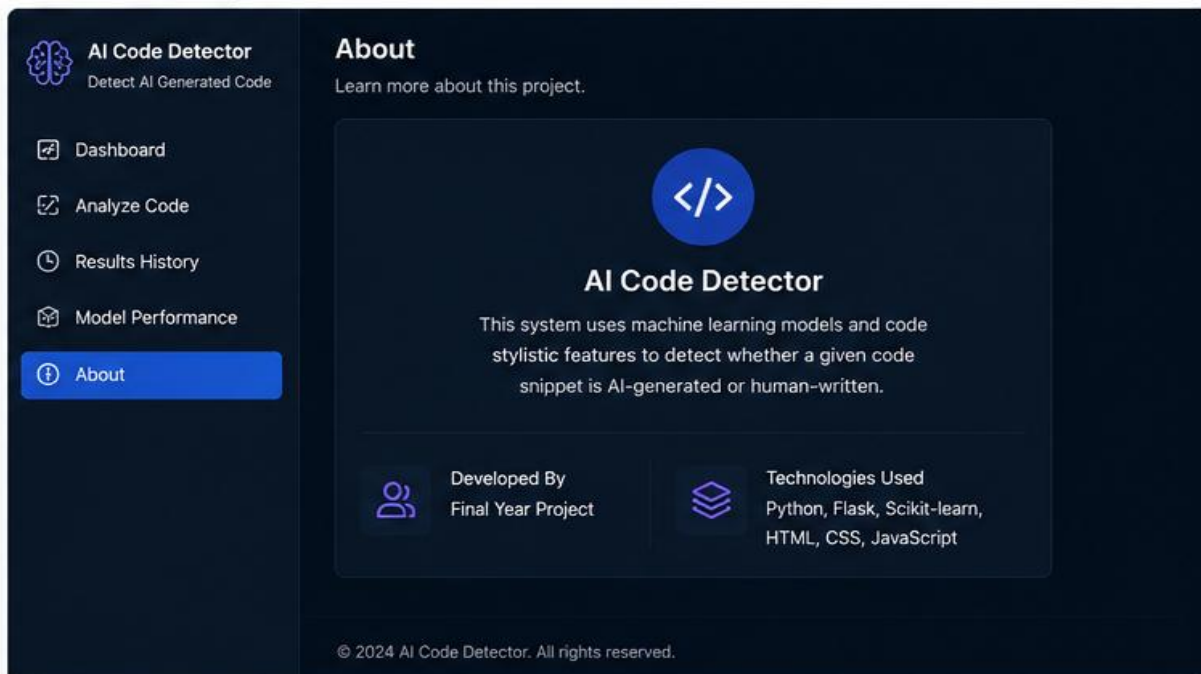


Fig.: 6.4.4

7. MAJOR OUTCOMES / RESULTS

<u>OUTCOME</u>	<u>BENEFIT</u>
Accurate Code Classification	Ensures fair academic evaluation
Authorship Identification	Verifies code ownership and originality
Improved Plagiarism Detection	Reduces misuse of AI tools
Software Authenticity Assurance	Strengthens reliability in codebases
Feature-Based Analysis	Provides explainable and transparent results
Enhanced Transparency	Builds trust in evaluation and decision-making

Table: 7.1

8. CONCLUSION

The development of this framework successfully bridges the “*authenticity gap*” created by modern Large Language Models. By quantifying the “Digital DNA” of source code, we have moved beyond simple text matching to a more profound understanding of authorship. Our findings confirm that while AI synthesizes functionally perfect code, it consistently lacks the idiosyncratic “noise” inherent in human thought. This “Noise vs. Perfection” principle provides a robust foundation for maintaining integrity in both *classrooms* and *corporations*.

The integration of AST structural analysis ensures that our classification is not just accurate, but fully interpretable. We have proven that the subtle “scuff marks” of human creativity are just as measurable as the symmetry of a machine. This system acts as a vital sensory layer, protecting original work in an increasingly automated world. As AI models continue to evolve, our scalable methodology offers a necessary defence for software accountability. Ultimately, this project preserves the standard of truth by valuing the unique fingerprints of human developers. It is a decisive step toward a more transparent, trustworthy, and authentic digital ecosystem.

9. FUTURE SCOPE

- a) *Multi-Language Support*: Expanding detection to Java, C++, and JavaScript.
- b) *IDE Integration*: Developing a VS Code extension for real-time authorship verification.
- c) *Adversarial Defence*: Training the model to detect AI code that has been intentionally “humanized” with artificial errors.

10. REFERENCES

- a. **Scikit-learn Documentation**
<https://scikit-learn.org/stable/>
- b. **Python AST Module Documentation**
<https://docs.python.org/3/library/ast.html>
- c. **Python Official Documentation**
<https://docs.python.org/3/>
- d. **Research on Code Authorship Attribution (Stylometry)**
Caliskan-Islam et al., “De-anonymizing Programmers via Code Stylometry”
<https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-caliskan-islam.pdf>
- e. **AI Code Detection Research**
<https://arxiv.org/search/?query=AI+generated+code+detection&searchtype=all>
- f. **Survey on Machine Learning for Code Analysis**
<https://arxiv.org/abs/2006.12106>
- g. **Research on Software Forensics & Code Analysis**
<https://ieeexplore.ieee.org/document/7876926>